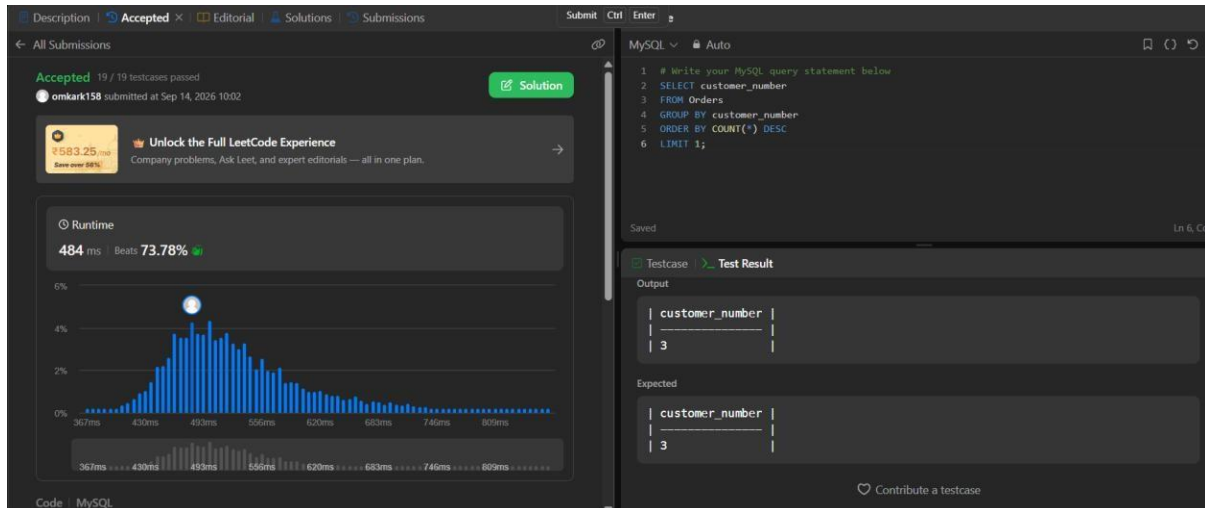


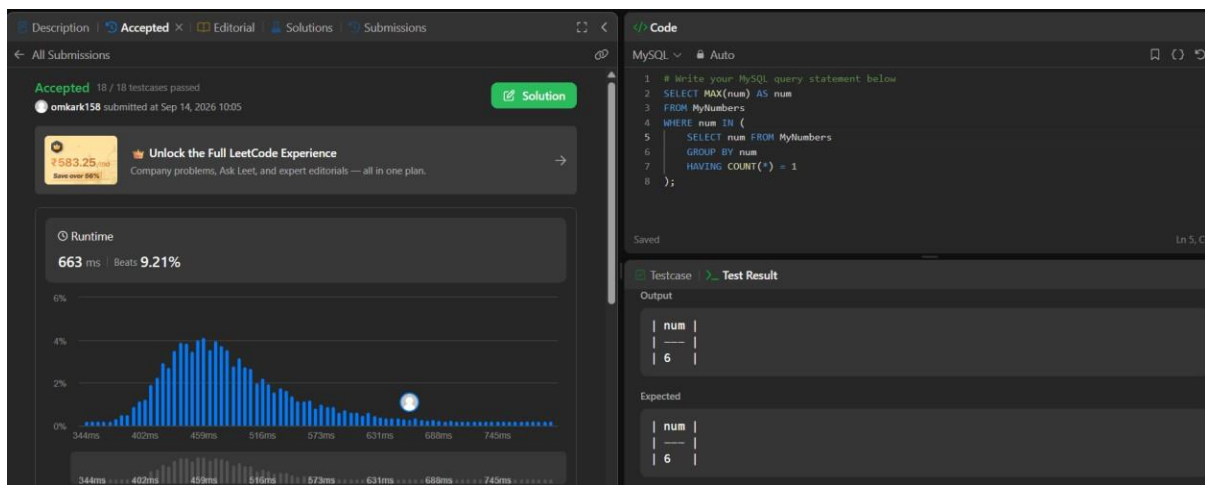
DBT SELF LEARNING

GROUP BY PROBLEMS

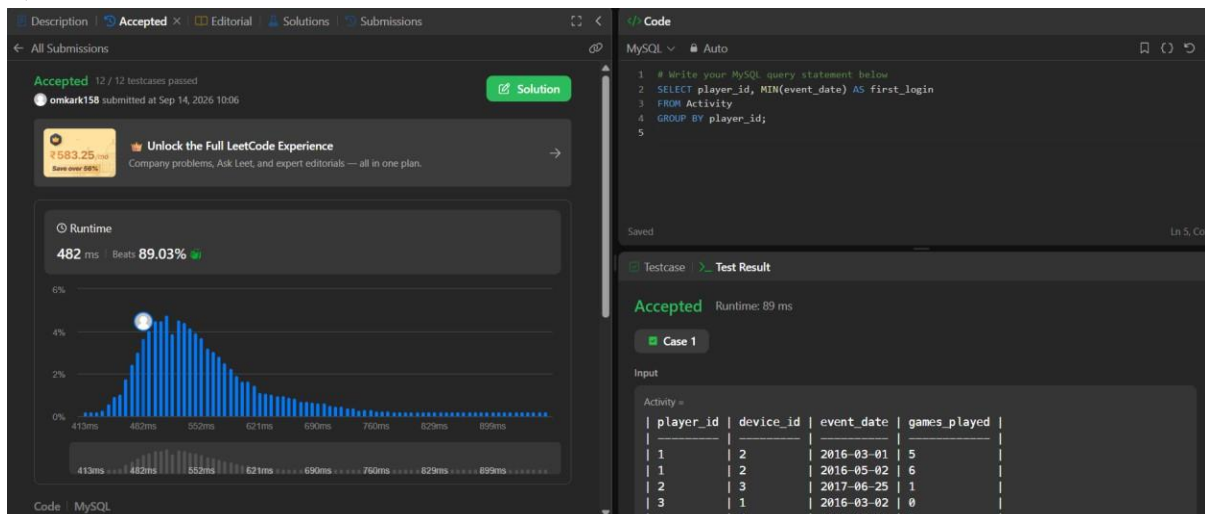
1)



2)



3)



4)

Description

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted 22 / 22 testcases passed

omkark158 submitted at Sep 14, 2026 10:09

583.25ms

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editors — all in one plan.

Runtime

562 ms | Beats: 75.74%

Code MySQL

Code

MySQL Auto

```

1 # Write your MySQL query statement below
2 SELECT
3     date_id, make_name,
4     COUNT(DISTINCT lead_id) AS unique_leads,
5     COUNT(DISTINCT partner_id) AS unique_partners
6 FROM DailySales
7 GROUP BY date_id, make_name;

```

Saved

Ln 7, Col 1

Testcase Test Result

Accepted Runtime: 70 ms

Case 1

Input

DailySales =

date_id	make_name	lead_id	partner_id
2020-12-8	toyota	0	1
2020-12-8	toyota	1	0
2020-12-8	toyota	1	2
2020-12-7	toyota	0	2

5)

Description

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted 22 / 22 testcases passed

omkark158 submitted at Sep 14, 2026 10:11

583.25ms

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editors — all in one plan.

Runtime

591 ms | Beats: 53.72%

Code MySQL

Code

MySQL Auto

```

1 SELECT emp_id, event_day AS day, SUM(out_time - in_time) AS total_time
2 FROM Employees
3 GROUP BY emp_id, event_day;
4

```

Saved

Ln 3, Col 1

Testcase Test Result

Accepted Runtime: 96 ms

Case 1

Input

Employees =

emp_id	event_day	in_time	out_time
1	2020-11-28	4	32
1	2020-11-28	55	200
1	2020-12-3	1	42

6)

Description

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted 12 / 12 testcases passed

omkark158 submitted at Sep 14, 2026 10:12

583.25ms

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editors — all in one plan.

Runtime

353 ms | Beats: 93.00%

Code MySQL

Code

MySQL Auto

```

1 # Write your MySQL query statement below
2 SELECT actor_id, director_id
3 FROM ActorDirector
4 GROUP BY actor_id, director_id
5 HAVING COUNT(*) >= 3;

```

Saved

Ln 5, Col 1

Testcase Test Result

Accepted Runtime: 66 ms

Case 1

Input

ActorDirector =

actor_id	director_id	timestamp
1	1	0
1	1	1
1	1	2
1	2	3
1	2	4

7)

Description
Accepted
Editorial
Solutions
Submissions

All Submissions

Accepted 24 / 24 testcases passed
omark158 submitted at Sep 14, 2026 10:17

Solution

1583.25ms
Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime
601 ms Beats 95.42%

Code: MySQL

Code
MySQL Auto

```

1 # Write your MySQL query statement below
2 SELECT user_id, MAX(time_stamp) AS last_stamp
3 FROM Logins
4 WHERE YEAR(time_stamp) = 2020
5 GROUP BY user_id;
6

```

Saved
Ln 6, Col

Testcase
Test Result

Accepted Runtime: 106 ms

Case 1

Input

Logins =

user_id	time_stamp
6	2020-06-30 15:06:07
6	2021-04-21 14:06:06
6	2019-03-07 00:18:15
8	2020-02-01 05:10:53

8)

Description
Accepted
Editorial
Solutions
Submissions

All Submissions

Accepted 13 / 13 testcases passed
omark158 submitted at Sep 14, 2026 10:19

Solution

1583.25ms
Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime
328 ms Beats 98.73%

Code: MySQL

Code
MySQL Auto

```

1 # Write your MySQL query statement below
2 SELECT
3     query_name,
4     ROUND(AVG(rating / position), 2) AS quality,
5     ROUND(AVG(rating < 3) * 100, 2) AS poor_query_percentage
6 FROM Queries
7 GROUP BY query_name;

```

Saved
Ln 7, Col

Testcase
Test Result

Accepted Runtime: 69 ms

Case 1

Input

Queries =

query_name	result	position	rating
Dog	Golden Retriever	1	5
Dog	German Shepherd	2	5
Dog	Mule	200	1
Cat	Shirazi	5	2

9)

Description
Accepted
Editorial
Solutions
Submissions

All Submissions

Accepted 17 / 17 testcases passed
omark158 submitted at Sep 14, 2026 10:21

Solution

1583.25ms
Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime
536 ms Beats 57.45%

Code: MySQL

Code
MySQL Auto

```

1 # Write your MySQL query statement below
2 SELECT activity_date AS day,
3        COUNT(DISTINCT user_id) AS active_users
4 FROM Activity
5 WHERE activity_date BETWEEN '2019-06-28' AND '2019-07-27'
6 GROUP BY activity_date;

```

Saved
Ln 6, Col

Testcase
Test Result

Accepted Runtime: 123 ms

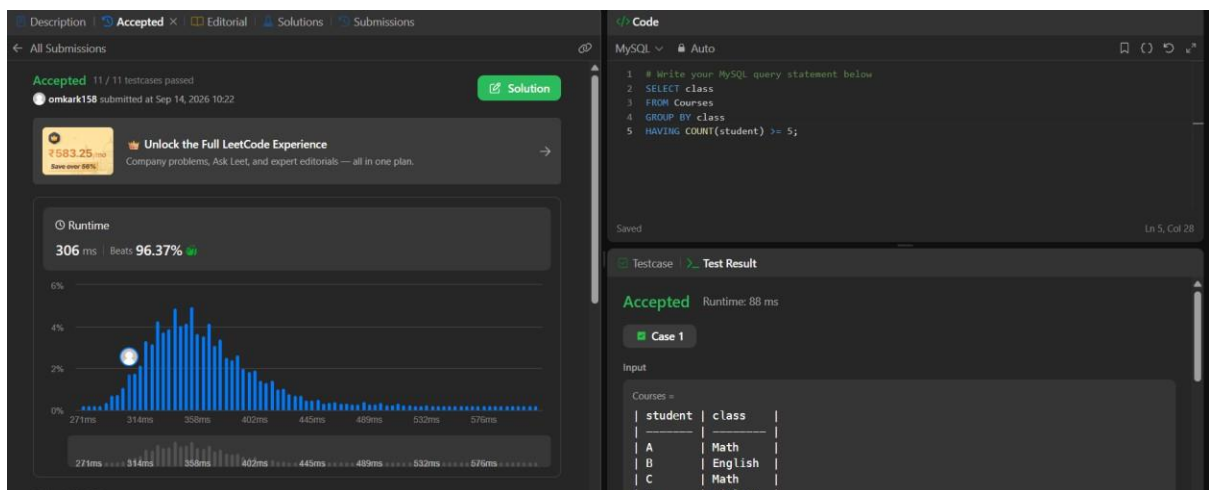
Case 1

Input

Activity =

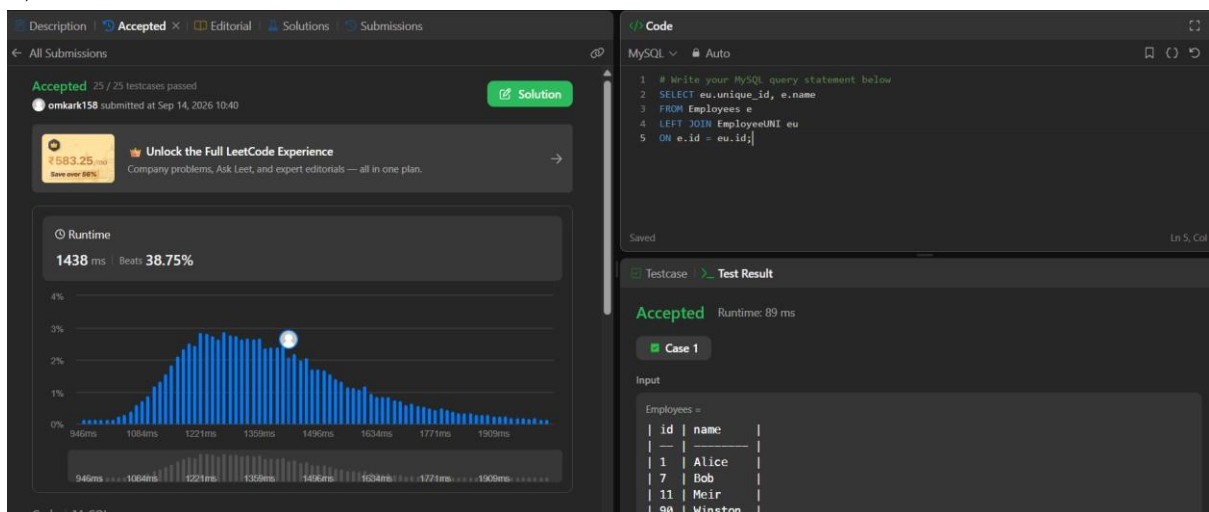
user_id	session_id	activity_date	activity_type
1	1	2019-07-20	open_session
1	1	2019-07-20	scroll_down
1	1	2019-07-20	end_session
2	4	2019-07-20	open_session

10)

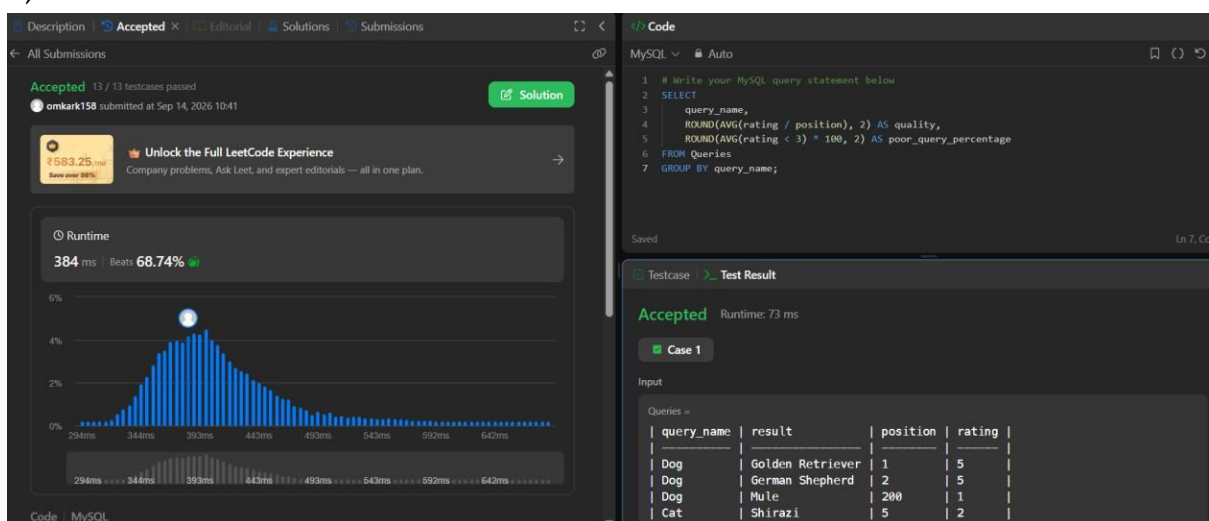


JOIN PROBLEMS

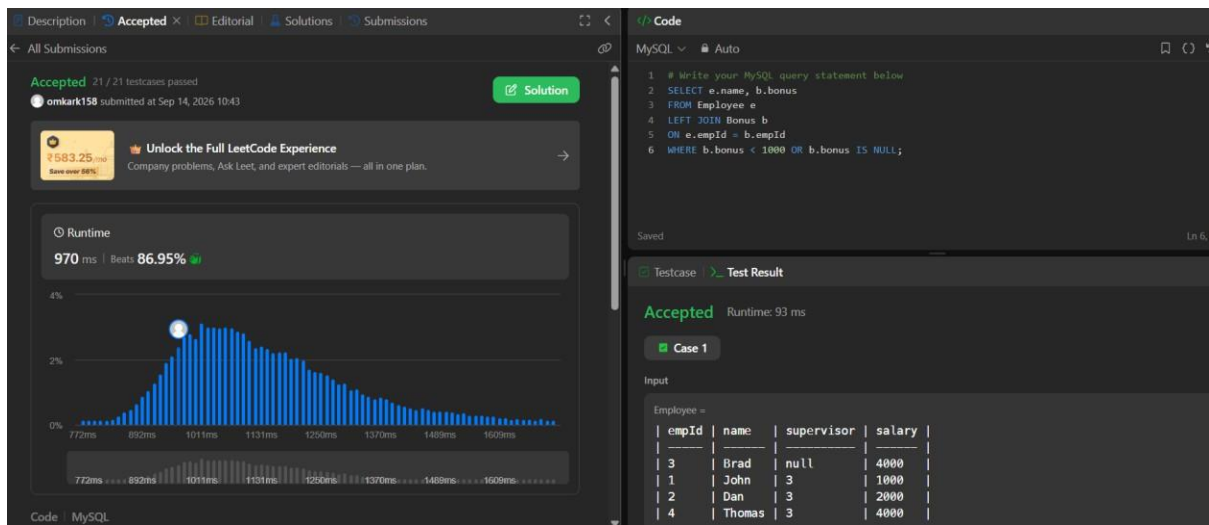
1)



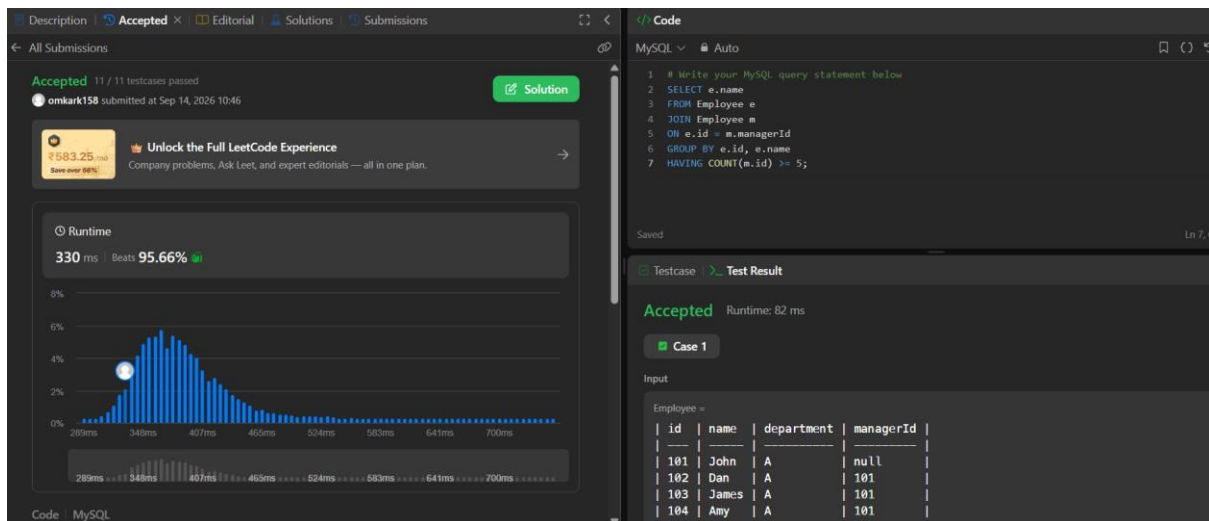
2)



3)



5)



6)

Your Population Census submission got 10.00 points. [Share](#) [Post](#)

You are now 70 points away from the 1st star for your sql badge.

[Try the next challenge](#) | [Try a Random Challenge](#)

Problem | Submissions | Leaderboard | Discussions

Given the **CITY** and **COUNTRY** tables, query the sum of the populations of all cities where the CONTINENT is 'Asia'.

Note: CITY.CountryCode and COUNTRY.Code are matching key columns.

Input Format



The **CITY** and **COUNTRY** tables are described as follows:



Change Theme Language MySQL

```
1 SELECT SUM(CITY.POPULATION)
2 FROM CITY
3 JOIN COUNTRY
4 ON CITY.CountryCode = COUNTRY.Code
5 WHERE COUNTRY.Continent = 'Asia';
```

7)

Your Average Population of Each Continent submission got 10.00 points. [Share](#) [Post](#)

You are now 60 points away from the 1st star for your sql badge.

[Try the next challenge](#) | [Try a Random Challenge](#)

Problem | Submissions | Leaderboard | Discussions

Given the **CITY** and **COUNTRY** tables, query the names of all the continents (COUNTRY.Continent) and their respective average city populations (CITY.Population) rounded down to the nearest integer.

Note: CITY.CountryCode and COUNTRY.Code are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:




Change Theme Language MySQL

```

1 SELECT COUNTRY.CONTINENT, FLOOR(AVG(CITY.POPULATION))
2 FROM CITY
3 JOIN COUNTRY
4 ON CITY.COUNTRYCODE = COUNTRY.CODE
5 GROUP BY COUNTRY.CONTINENT;

```

9)

Description | Accepted x | Editorial | Solutions | Submissions

← All Submissions

Accepted 16 / 16 testcases passed

omark158 submitted at Sep 14, 2026 10:52

[Solution](#)

Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime
478 ms | Beats 85.25%



Code | MySQL

MySQL | Auto

```

1 # Write your MySQL query statement below
2 SELECT product_id, 10 AS price
3 FROM Products
4 GROUP BY product_id
5 HAVING MIN(change_date) > '2019-08-16'
6
7 UNION
8
9 SELECT product_id, new_price AS price
10 FROM Products

```

Saved Ln 17, Col 1

Testcase | Test Result

Accepted Runtime: 70 ms

Case 1

Input

product_id	new_price	change_date
1	20	2019-08-14
2	50	2019-08-14
1	30	2019-08-15
1	35	2019-08-16
2	55	2019-08-17

SUBQUERY PROBLEMS

1)

Description | Editorial | Solutions | Submissions

181. Employees Earning More Than Their Managers

Solved

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: Employee

Column Name	Type
id	int
name	varchar
salary	int
managerId	int

id is the primary key (column with unique values) for this table.
Each row of this table indicates the ID of an employee, their name, salary, and the ID of their manager.

Write a solution to find the employees who earn more than their managers.

Return the result table in **any order**.

The result format is in the following example.

Code

MySQL | Auto

```
1 # Write your MySQL query statement below
2 SELECT e.name AS Employee
3 FROM Employee e
4 JOIN Employee m
5 ON e.managerId = m.id
6 WHERE e.salary > m.salary;
```

Saved

Ln 6, Col 1

Testcase | Test Result

Accepted Runtime: 66 ms

Case 1

Input

Employee =

id	name	salary	managerId
1	Joe	70000	3
2	Henry	80000	4
3	Sam	60000	null

2)

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 12 / 12 testcases passed

omikark158 submitted at Sep 14, 2026 10:56

Solution

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

606 ms | Beat: 62.77%

451ms 524ms 597ms 670ms 743ms 816ms 889ms 962ms

Code

MySQL | Auto

```
1 # Write your MySQL query statement below
2 SELECT c.name AS Customers
3 FROM Customers c
4 LEFT JOIN Orders o
5 ON c.id = o.customerId
6 WHERE o.customerId IS NULL;
```

Saved

Ln 6, Col 1

Testcase | Test Result

Accepted Runtime: 96 ms

Case 1

Input

Customers =

id	name
1	Joe
2	Henry
3	Sam
4	Max

3)

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 15 / 15 testcases passed

omikark158 submitted at Sep 14, 2026 10:57

Solution

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

513 ms | Beat: 60.90%

322ms 407ms 493ms 579ms 664ms 750ms 836ms 921ms

Code

MySQL | Auto

```
1 # Write your MySQL query statement below
2 SELECT w1.id
3 FROM Weather w1
4 JOIN Weather w2
5 ON DATEDIFF(w1.recordDate, w2.recordDate) = 1
6 WHERE w1.temperature > w2.temperature;
```

Saved

Ln 6, Col 1

Testcase | Test Result

Accepted Runtime: 62 ms

Case 1

Input

Weather =

id	recordDate	temperature
1	2015-01-01	10
2	2015-01-02	25
3	2015-01-03	20

4)

Description
Accepted
Editorial
Solutions
Submissions

All Submissions

Accepted 21 / 21 testcases passed
omark158 submitted at Sep 14, 2026 10:58

Solution

Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime
1722 ms Beats 36.18%

Code MySQL

Code
MySQL
Auto

```

3 FROM SalesPerson
4 WHERE sales_id NOT IN (
5     SELECT o.sales_id
6 FROM Orders o
7 JOIN Company c
8 ON o.com_id = c.com_id
9 WHERE c.name = 'RED'
10 );
11

```

Saved
Ln 11, Col 1

Testcase
Test Result

Accepted Runtime: 141 ms

Case 1

Input

SalesPerson =

sales_id	name	salary	commission_rate	hire_date
1	John	100000	6	4/1/2006
2	Amy	12000	5	5/1/2010
3	Mark	65000	12	12/25/2008
4	Pam	25000	25	1/1/2005
5	Alex	30000	10	2/3/2007

5)

Description
Accepted
Editorial
Solutions
Submissions

All Submissions

Accepted 23 / 23 testcases passed
omark158 submitted at Sep 14, 2026 10:59

Solution

Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime
558 ms Beats 85.78%

Code MySQL

Code
MySQL
Auto

```

1 # Write your MySQL query statement below
2 SELECT DISTINCT l1.num AS ConsecutiveNums
3 FROM Logs l1
4 JOIN Logs l2 ON l1.id = l2.id - 1
5 JOIN Logs l3 ON l2.id = l3.id - 1
6 WHERE l1.num = l2.num
7 AND l2.num = l3.num;

```

Saved
Ln 7, Col 21

Testcase
Test Result

Accepted Runtime: 66 ms

Case 1

Input

Logs =

id	num
1	1
2	1
3	1
4	2

6)

Description
Accepted
Editorial
Solutions
Submissions

All Submissions

Accepted 8 / 8 testcases passed
omark158 submitted at Sep 14, 2026 11:02

Solution

Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime
459 ms Beats 51.80%

Code MySQL

Code
MySQL
Auto

```

1 # Write your MySQL query statement below
2 SELECT p.firstName, p.lastName, a.city, a.state
3 FROM Person p
4 LEFT JOIN Address a
5 ON p.personId = a.personId;

```

Saved
Ln 5, Col 8

Testcase
Test Result

Accepted Runtime: 90 ms

Case 1

Input

Person =

personId	lastName	firstName
1	Wang	Allen
2	Alice	Bob

7)

Description

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted 11 / 11 testcases passed

omark158 submitted at Sep 14, 2026 11:06

7583.25ms

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

449 ms | Beats: 19.28%

Code

MySQL

```

1 # Write your MySQL query statement below
2 SELECT score,
3       DENSE_RANK() OVER (ORDER BY score DESC) AS 'rank'
4 FROM Scores
5 ORDER BY score DESC;

```

Testcase

Test Result

Accepted Runtime: 67 ms

Case 1

Input

Scores =

id	score
1	3.5
2	3.65
3	4
4	3.85

8)

Description

Editorial

Solutions

Submissions

570. Managers with at Least 5 Direct Reports

Medium

Topics

Companies

Hint

SQL Schema

Pandas Schema

Table: Employee

Column Name	Type
id	int
name	varchar
department	varchar
managerId	int

id is the primary key (column with unique values) for this table. Each row of this table indicates the name of an employee, their department, and the id of their manager. If managerId is null, then the employee does not have a manager. No employee will be the manager of himself.

Write a solution to find managers with at least **five** direct reports.

Return the result table in **any order**.

The result format is in the following example.

Code

MySQL

```

1 # Write your MySQL query statement below
2 SELECT e.name
3 FROM Employee e
4 JOIN Employee m
5 ON e.id = m.managerId
6 GROUP BY e.id, e.name
7 HAVING COUNT(m.id) >= 5;

```

Testcase

Test Result

Accepted Runtime: 64 ms

Case 1

Input

Employee =

id	name	department	managerId
101	John	A	null
102	Dan	A	101
103	James	A	101
104	Amy	A	101

9)

Description

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted 22 / 22 testcases passed

omark158 submitted at Sep 14, 2026 11:11

7583.25ms

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

704 ms | Beats: 90.07%

Code

MySQL

```

1 # Write your MySQL query statement below
2 DELETE FROM Person
3 WHERE id NOT IN (
4     SELECT id
5     FROM (
6         SELECT MIN(id) AS id
7         FROM Person
8         GROUP BY email
9     ) AS temp
10 );

```

Testcase

Test Result

Accepted Runtime: 91 ms

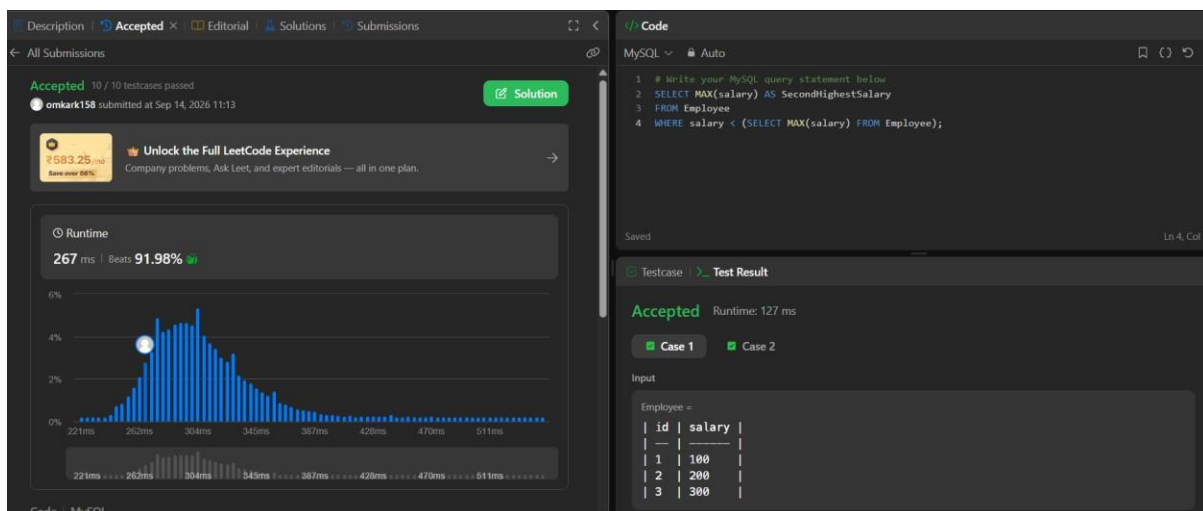
Case 1

Input

Person =

id	email
1	john@example.com
2	bob@example.com
3	john@example.com

11)



MongoDB -

1. Practice CRUD operations on Mongo. 1. Create two collections -- emp and dept. 2. Insert the emp and dept records. 3. Increase salary of dept 10 emps by 100. 4. Display emp name, job, deptno, sal in sorted order of deptno, job, sal (desc) for all emps whose sal more than 1000. 5. Delete the dept with id 40.

2. Learn grouping operations using Mongo aggregation pipeline 1. Display job wise total sal, max sal, min sal, and avg sal for all jobs other than PRESIDENT. 2. Display emp name, dept name, income (sal + comm) in sorted order of their income.

```
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.

C:\Users\91879>cmd
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.

C:\Users\91879>mongosh
Current Mongosh Log ID: 6aaa8ae6e80df6ab8686b01c
Connecting to:  mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.3.2
Using MongoDB:  8.2.5
Using Mongosh:  2.3.2
mongosh 2.10.0 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-09-12T08:24:09.550+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----
```

11)

```
test> use dbt_learning
switched to db dbt_learning
dbt_learning> db.dept.insertMany([
...   {
...     deptno: 10,
...     dname: "ACCOUNTING",
...     loc: "NEW YORK"
...   },
...   {
...     deptno: 20,
...     dname: "RESEARCH",
...     loc: "DALLAS"
...   },
...   {
...     deptno: 30,
...     dname: "SALES",
...     loc: "CHICAGO"
...   },
...   {
...     deptno: 40,
...     dname: "OPERATIONS",
...     loc: "BOSTON"
...   }
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6aaa8bc2e80df6ab8686b01d'),
    '1': ObjectId('6aaa8bc2e80df6ab8686b01e'),
    '2': ObjectId('6aaa8bc2e80df6ab8686b01f'),
    '3': ObjectId('6aaa8bc2e80df6ab8686b020')
  }
}
dbt_learning> db.dept.find()
[
  {
    _id: ObjectId('6aaa8bc2e80df6ab8686b01d'),
    deptno: 10,
    dname: 'ACCOUNTING',
    loc: 'NEW YORK'
  },

```

```
    _id: ObjectId('6aaa8bc2e80df6ab8686b01e'),
    deptno: 20,
    dname: 'RESEARCH',
    loc: 'DALLAS'
  },
  {
    _id: ObjectId('6aaa8bc2e80df6ab8686b01f'),
    deptno: 30,
    dname: 'SALES',
    loc: 'CHICAGO'
  },
  {
    _id: ObjectId('6aaa8bc2e80df6ab8686b020'),
    deptno: 40,
    dname: 'OPERATIONS',
    loc: 'BOSTON'
  }
]
dbt_learning> db.emp.insertMany([
...   {
...     empno: 7369,
...     ename: "SMITH",
...     job: "CLERK",
...     deptno: 20,
...     sal: 800,
...     comm: 0
...   },
...   {
...     empno: 7499,
...     ename: "ALLEN",
...     job: "SALESMAN",
...     deptno: 30,
...     sal: 1600,
...     comm: 300
...   },
...   {
...     empno: 7521,
...     ename: "WARD",
...     job: "SALESMAN",
...     deptno: 30,
...     sal: 1250,

```

11)

```
...      ename: "KING",
...      job: "PRESIDENT",
...      deptno: 10,
...      sal: 5000,
...      comm: 0
...    },
...    {
...      empno: 7844,
...      ename: "TURNER",
...      job: "SALESMAN",
...      deptno: 30,
...      sal: 1500,
...      comm: 0
...    },
...    {
...      empno: 7876,
...      ename: "ADAMS",
...      job: "CLERK",
...      deptno: 20,
...      sal: 1100,
...      comm: 0
...    },
...    {
...      empno: 7900,
...      ename: "JAMES",
...      job: "CLERK",
...      deptno: 30,
...      sal: 950,
...      comm: 0
...    },
...    {
...      empno: 7902,
...      ename: "FORD",
...      job: "ANALYST",
...      deptno: 20,
...      sal: 3000,
...      comm: 0
...    },
...    {
...      empno: 7934,
...      ename: "MILLER",
...    }
  ],
}
```

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6aaa8be5e80df6ab8686b021'),
    '1': ObjectId('6aaa8be5e80df6ab8686b022'),
    '2': ObjectId('6aaa8be5e80df6ab8686b023'),
    '3': ObjectId('6aaa8be5e80df6ab8686b024'),
    '4': ObjectId('6aaa8be5e80df6ab8686b025'),
    '5': ObjectId('6aaa8be5e80df6ab8686b026'),
    '6': ObjectId('6aaa8be5e80df6ab8686b027'),
    '7': ObjectId('6aaa8be5e80df6ab8686b028'),
    '8': ObjectId('6aaa8be5e80df6ab8686b029'),
    '9': ObjectId('6aaa8be5e80df6ab8686b02a'),
    '10': ObjectId('6aaa8be5e80df6ab8686b02b'),
    '11': ObjectId('6aaa8be5e80df6ab8686b02c'),
    '12': ObjectId('6aaa8be5e80df6ab8686b02d'),
    '13': ObjectId('6aaa8be5e80df6ab8686b02e')
  }
}
{ deptno: 10 },
... { $inc: { sal: 100 } }
... )
Uncaught:
SyntaxError: Unexpected token (1:18)

> 1 | { deptno: 10 },
  |   ^
  2 | { $inc: { sal: 100 } }
  3 | )
  4 |

dbt_learning> db.EMP.updateMany(
...   { deptno: 10 },
...   { $inc: { sal: 100 } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
```

11)

```
dbt_learning> db.emp.find(
...   { deptno: 10 },
...   { _id: 0, ename: 1, deptno: 1, sal: 1 }
... )
[
  { ename: 'CLARK', deptno: 10, sal: 2550 },
  { ename: 'KING', deptno: 10, sal: 5100 },
  { ename: 'MILLER', deptno: 10, sal: 1400 }
]
dbt_learning> db.emp.find(
...   { sal: { $gt: 1000 } },
...   {
...     _id: 0,
...     ename: 1,
...     job: 1,
...     deptno: 1,
...     sal: 1
...   }
... ).sort({
...   deptno: 1,
...   job: 1,
...   sal: -1
... })
[
  { ename: 'MILLER', job: 'CLERK', deptno: 10, sal: 1400 },
  { ename: 'CLARK', job: 'MANAGER', deptno: 10, sal: 2550 },
  { ename: 'KING', job: 'PRESIDENT', deptno: 10, sal: 5100 },
  { ename: 'SCOTT', job: 'ANALYST', deptno: 20, sal: 3000 },
  { ename: 'FORD', job: 'ANALYST', deptno: 20, sal: 3000 },
  { ename: 'ADAMS', job: 'CLERK', deptno: 20, sal: 1100 },
  { ename: 'JONES', job: 'MANAGER', deptno: 20, sal: 2975 },
  { ename: 'BLAKE', job: 'MANAGER', deptno: 30, sal: 2850 },
  { ename: 'ALLEN', job: 'SALESMAN', deptno: 30, sal: 1600 },
  { ename: 'TURNER', job: 'SALESMAN', deptno: 30, sal: 1500 },
  { ename: 'WARD', job: 'SALESMAN', deptno: 30, sal: 1250 },
  { ename: 'MARTIN', job: 'SALESMAN', deptno: 30, sal: 1250 }
]
dbt_learning> db.dept.deleteOne({
...   deptno: 40
... })
```

```
dbt_learning> db.dept.deleteOne({
...   deptno: 40
... })
{ acknowledged: true, deletedCount: 1 }
dbt_learning> db.dept.find()
[
  {
    _id: ObjectId('6aaa8bc2e80df6ab8686b01d'),
    deptno: 10,
    dname: 'ACCOUNTING',
    loc: 'NEW YORK'
  },
  {
    _id: ObjectId('6aaa8bc2e80df6ab8686b01e'),
    deptno: 20,
    dname: 'RESEARCH',
    loc: 'DALLAS'
  },
  {
    _id: ObjectId('6aaa8bc2e80df6ab8686b01f'),
    deptno: 30,
    dname: 'SALES',
    loc: 'CHICAGO'
  }
]
dbt_learning> db.emp.aggregate([
...   {
...     $match: {
...       job: { $ne: "PRESIDENT" }
...     },
...   },
...   {
...     $group: {
...       _id: "$job",
...       total_sal: { $sum: "$sal" },
...       max_sal: { $max: "$sal" },
...       min_sal: { $min: "$sal" },
...       avg_sal: { $avg: "$sal" }
...     }
...   }
... ])
```

11)

```
[
  {
    _id: 'MANAGER',
    total_sal: 8375,
    max_sal: 2975,
    min_sal: 2550,
    avg_sal: 2791.6666666666665
  },
  {
    _id: 'CLERK',
    total_sal: 4250,
    max_sal: 1400,
    min_sal: 800,
    avg_sal: 1062.5
  },
  {
    _id: 'SALESMAN',
    total_sal: 5600,
    max_sal: 1600,
    min_sal: 1250,
    avg_sal: 1400
  },
  {
    _id: 'ANALYST',
    total_sal: 6000,
    max_sal: 3000,
    min_sal: 3000,
    avg_sal: 3000
  }
]
dbt_learning> db.emp.aggregate([
...   {
...     $lookup: {
...       from: "dept",
...       localField: "deptno",
...       foreignField: "deptno",
...       as: "department"
...     }
...   },
...   {
...     {
...       income: {
...         $add: ["$sal", "$comm"]
...       }
...     }
...   },
...   {
...     $sort: {
...       income: 1
...     }
...   }
... ])
[
  { emp_name: 'SMITH', dept_name: 'RESEARCH', income: 800 },
  { emp_name: 'JAMES', dept_name: 'SALES', income: 950 },
  { emp_name: 'ADAMS', dept_name: 'RESEARCH', income: 1100 },
  { emp_name: 'MILLER', dept_name: 'ACCOUNTING', income: 1400 },
  { emp_name: 'TURNER', dept_name: 'SALES', income: 1500 },
  { emp_name: 'WARD', dept_name: 'SALES', income: 1750 },
  { emp_name: 'ALLEN', dept_name: 'SALES', income: 1900 },
  { emp_name: 'CLARK', dept_name: 'ACCOUNTING', income: 2550 },
  { emp_name: 'MARTIN', dept_name: 'SALES', income: 2650 },
  { emp_name: 'BLAKE', dept_name: 'SALES', income: 2850 },
  { emp_name: 'JONES', dept_name: 'RESEARCH', income: 2975 },
  { emp_name: 'SCOTT', dept_name: 'RESEARCH', income: 3000 },
  { emp_name: 'FORD', dept_name: 'RESEARCH', income: 3000 },
  { emp_name: 'KING', dept_name: 'ACCOUNTING', income: 5100 }
]
dbt_learning> |
```

```
...       income: {
...         $add: ["$sal", "$comm"]
...       }
...     }
...   },
...   {
...     $sort: {
...       income: 1
...     }
...   }
... ])
[
  { emp_name: 'SMITH', dept_name: 'RESEARCH', income: 800 },
  { emp_name: 'JAMES', dept_name: 'SALES', income: 950 },
  { emp_name: 'ADAMS', dept_name: 'RESEARCH', income: 1100 },
  { emp_name: 'MILLER', dept_name: 'ACCOUNTING', income: 1400 },
  { emp_name: 'TURNER', dept_name: 'SALES', income: 1500 },
  { emp_name: 'WARD', dept_name: 'SALES', income: 1750 },
  { emp_name: 'ALLEN', dept_name: 'SALES', income: 1900 },
  { emp_name: 'CLARK', dept_name: 'ACCOUNTING', income: 2550 },
  { emp_name: 'MARTIN', dept_name: 'SALES', income: 2650 },
  { emp_name: 'BLAKE', dept_name: 'SALES', income: 2850 },
  { emp_name: 'JONES', dept_name: 'RESEARCH', income: 2975 },
  { emp_name: 'SCOTT', dept_name: 'RESEARCH', income: 3000 },
  { emp_name: 'FORD', dept_name: 'RESEARCH', income: 3000 },
  { emp_name: 'KING', dept_name: 'ACCOUNTING', income: 5100 }
]
dbt_learning> |
```

CDAC Admission Process Case Study

Design database for CDAC Admission System. Create appropriate tables and insert sample records. Write a Stored Procedure for Round1 allocation. Write a Stored Procedure for Round2 allocation. Note: 1. Create ER diagram following Normalization process. 2. Study database constraints in depth: PRIMARY KEY, FOREIGN-KEY, UNIQUE, CHECK, DEFAULT Apply appropriate constraints in CDAC admission system. 3. Learn about composite keys and candidate keys Identify keys in CDAC admission system. 4. Explore AUTO_INCREMENT / SEQUENCES Use appropriate AUTO_INCREMENT keys in CDAC admission system. 5. Study control structures: IF-ELSE, CASE, LOOPS (WHILE, REPEAT, LOOP) Use appropriate control

11)

structures in CDAC admission system stored procedures. 6. Learn stored procedure parameters: IN, OUT, IN-OUT. Use appropriate parameters for CDAC admission system stored procedures. 7. Understand cursors and their usage in following stored procedures. Make appropriate use of cursor in CDAC admission system stored procedures. 8. Explore error handling in stored procedures Implement appropriate error handling in CDAC admission system stored procedures